AI Agents Setup Guide

A shared library of AI agent skills, tools, and automation. Your projects import from it via live `@`-references — when canon is updated, your projects pick up changes automatically on the next session.

Setup

Step 1 — Clone canon

```
git clone https://github.com/sunitghub/canon.git ~/Developer/canon
```

Step 2 — Run init (once)

```
cd ~/Developer/canon && ./skills.sh init
```

Or if you prefer it available from anywhere:

```
export SKILLS=~/Developer/canon   # add to ~/.zshrc to make permanent
$SKILLS/skills.sh init
```

This configures all installed agents in one shot:

Agent	What gets configured
Claude Code	Handoff + quality hooks merged into `~/.claude/settings.json`. RTK wired automatically if installed.
Codex	RTK wired into `~/.codex/AGENTS.md` (skipped if RTK absent).
Pi	Copies `extensions/pi/handoff.ts` to `~/.pi/agent/extensions/`

Agents not installed are skipped. Re-run any time you move or rename the canon folder.

On success, `init` prints the available commands. You're ready to register skills.

> **RTK** (optional, recommended) — filters verbose CLI output, saving 60–90% of tokens on common operations. Install before running `init` so it gets wired automatically.

```
brew install rtk    # macOS
cargo install rtk   # WSL / Linux
```

Step 3 — Register skills in your project

The same command works for new and existing projects — `addall` merges into existing config files safely.

```
cd /path/to/your-project
$SKILLS/skills.sh addall        # register all available skills
        (recommended)

# Or pick individually:
$SKILLS/skills.sh add sprint    # full dev workflow (includes
        everything)
$SKILLS/skills.sh add pdf       # PDF read/extract/merge/split
```

Then verify:

```
$SKILLS/skills.sh status
```

Existing project only — `CLAUDE.md` and `AGENTS.md` are extended with `@`-imports, existing content is preserved. If you have prior architectural decisions worth keeping, add them to `DECISIONS.md` manually before the first sprint using the format in the How it works section; sprint will append from there. `HANDOFF.md` and `DECISIONS.md` are both created automatically on the first `sprint start` if they don't exist.

Here's what gets created and by whom:

	New project	Existing project, first sprint	Existing project, ongoing
Codebase	Agent proposes structure	Agent scans existing code	Agent scans existing code
DECISIONS.md	Automatic — first `sprint start`	Automatic — first `sprint start`	Automatic — sprint reads + appends
HANDOFF.md	Automatic — first `sprint start`	Automatic — first `sprint start`	Automatic — hooks + sprint

---

Using canon in an existing project

This example shows canon in an ongoing project — `DECISIONS.md` and `HANDOFF.md` already exist and have content. That's the state after a few sprints, and it's where the continuity payoff is most visible.

First sprint on an existing project? Neither file exists yet — same as a new project. The difference is the codebase is already there to scan. The agent creates both files during that first sprint, and from sprint two onward the example below is what you get.

Everything in **bold** is something you type. Everything else is automatic.

Opening the session

You open Claude Code. Before your first message reaches the agent:

> `handoff-inject` fires — reads `HANDOFF.md` silently into the prompt, once per 4-hour window.

The agent wakes up knowing the project state, prior decisions, and any mid-session discoveries — without you saying a word.

You: "Sprint start — add rate limiting to the login endpoint"

The agent:

1. Creates ticket `t-r4t3` and marks it in progress
2. Creates `.tickets/t-r4t3/blueprint.md` and `acceptance.md`
3. Reads `DECISIONS.md` — finds: *"Redis chosen for session state"*
4. Reads `HANDOFF.md` — picks up any open context from the last session
5. Does a targeted scan of affected files (auth/, login-related code, tests), then produces a sprint brief and waits:

```
Sprint t-r4t3 — Add rate limiting to login endpoint

Goal: Block brute-force attempts by tracking failed logins per IP
in Redis.

Files to modify:
  auth/views.py       — add rate limit check to login endpoint
  auth/middleware.py  — new @rate_limit decorator

Files to create:
  tests/test_rate_limit.py

Acceptance criteria:
  ✓ Login returns 429 after 5 failed attempts from the same IP
  ✓ Counter resets after 15 minutes
  ✓ Rate limit is not bypassable via header manipulation
  ✓ Tests pass

Constraint from DECISIONS.md: use Redis (already a dependency).

Ready to proceed?
```

You: "Yes"

The agent writes code. While reading the Redis client config, it notices the connection pool is set to 5.

> `capture` fires automatically: Appends to `HANDOFF.md` under `## Discoveries`: *"Redis connection pool capped at 5 — rate limit checks may queue under sustained login load. See `config/redis.py:12`."*

You didn't ask for this. It happened because the agent found a non-obvious constraint.

You: "Sprint complete"

The agent:

1. **Wrapup pipeline** — runs on all files modified since sprint start:
   - `code-simplifier` — removes a redundant null check, clarifies a variable name
   - `code-reviewer` — flags a missing test for the header-manipulation bypass criterion
   - `security-review` — confirms no injection risk in the Redis key construction
2. **Acceptance check** — reviews `acceptance.md`:
   - ✓ 429 returned after 5 failures
   - ✓ Counter resets after 15 minutes
   - ✗ No test for header bypass (flagged by reviewer)

   > Stops. *"Criterion 3 not met — bypass test is missing. Adding it now."* Writes the test. Re-checks: all ✓.

3. Appends to `DECISIONS.md`, updates `HANDOFF.md` next steps, closes the ticket.

You close Claude Code.

> `auto-handoff` fires — appends a snapshot to `HANDOFF.md`: modified files, recent commits, active tickets.

Next session — or the next agent — reads the file and picks up exactly here.

Using canon in a new project

Everything in **bold** is something you type. Everything else is automatic.

You: "Sprint start — build a user authentication API"

No prior context exists. The agent works from your description alone.

The agent:

1. Creates ticket `t-a1b2` and marks it in progress
2. Creates `.tickets/t-a1b2/blueprint.md` and `acceptance.md`
3. Reads `DECISIONS.md` — file doesn't exist yet, creates it with an empty log table
4. Reads `HANDOFF.md` — no prior context, starts fresh
5. Produces a sprint brief and waits

There's no existing codebase to analyze, so the brief is a *proposal* — the agent derives file structure and acceptance criteria from the task description and common patterns. The user can redirect anything before approving:

```
Sprint t-a1b2 — Build user authentication API

Goal: User registration, login, and token-based session
management.

Files to create:
  auth/models.py      — User model
  auth/views.py       — register, login, logout endpoints
  auth/tokens.py      — JWT generation and validation
  tests/test_auth.py

Acceptance criteria:
  ✓ POST /register creates a user and returns a token
  ✓ POST /login returns a token for valid credentials, 401 for
invalid
  ✓ Token expires after 24 hours
  ✓ Tests pass

No prior decisions found — proceeding with standard patterns.

Ready to proceed?
```

You: "Yes, and use postgres for the DB"

> `capture` fires: appends to `HANDOFF.md` under `## Discoveries`: *"Postgres chosen for auth DB — user-specified."*

The agent writes code. When it hits the DB connection, it notices the default pool size of 10.

> `capture` fires again: *"Postgres pool size defaults to 10 — fine for current load but worth revisiting at scale. See `config/db.py:8`."*

You: "Sprint complete"

The agent:

1. **Wrapup pipeline** runs on all new files.
2. **Acceptance check** — all criteria pass.
3. **DECISIONS.md** — agent writes its first entry:

Date	Decision	Reason
2026-05-17	JWT for session tokens	Stateless — no session store needed at this scale
2026-05-17	Postgres for auth DB	User specified

4. **HANDOFF.md** — updated with next steps: *"Wire auth middleware into protected endpoints."*

5. Ticket closed.

You close Claude Code.

> `auto-handoff` fires — snapshots current git state to `HANDOFF.md`.

Next session, the agent reads `HANDOFF.md` and `DECISIONS.md` and picks up from *"Wire auth middleware into protected endpoints"* — as if it never left.

How it works

`sprint` encapsulates a full dev lifecycle in two commands. Everything underneath runs automatically.

```
sprint ──────────────────────────────── planned dev lifecycle
  │
  ├── PLAN
  │     tkt                track work, one ticket per sprint
  │     blueprint.md       files to touch, step-by-step build plan
  │     acceptance.md      binary definition of done
  │     DECISIONS.md       durable architectural decisions (repo
root)
  │
  ├── BUILD
  │     capture (auto)     non-obvious discoveries → HANDOFF.md
  │     efficiency         coding principles, always on, no
invocation
  │
  └── SHIP
        wrapup
          code-simplifier   clarity and redundancy pass
          code-reviewer     seven-dimension logic review
          security-review   high-confidence vulnerability scan

Session hooks (fire automatically):
  handoff-inject   session start → agent reads HANDOFF.md silently
  auto-handoff     session end   → agent snapshots git state to
HANDOFF.md
```

Layer 1 — Session continuity

The problem: AI agents start cold. Every new session — or context window exhaustion mid-session — means re-explaining where things stand.

What it does: `HANDOFF.md` is a git-tracked file in the project root. Two hooks automate its lifecycle:

- **`handoff-inject`** (session start) — injects `HANDOFF.md` into your first prompt, once per 4-hour window. The agent wakes up knowing where things stand without you saying a word.
- **`auto-handoff`** (session end) — appends a timestamped snapshot: modified files, recent commits, active tickets. Safety net when context runs out mid-session.

```
# Handoff

## Current Focus
One sentence — what are we working on.

## In Progress
- file/path or ticket-id: what's started but not finished

## Recent Decisions
- Decision and WHY (not what — the diff shows what)

## Discoveries
- Non-obvious facts found through investigation

## Next Steps
1. First concrete next action
```

Layer 2 — Knowledge capture

The problem: Agents discover non-obvious constraints mid-session — a connection pool cap, a config quirk, an edge case only found by running code. Without recording them immediately, they're lost when context compacts or the session ends.

What it does: `capture` writes discoveries to `HANDOFF.md ## Discoveries` the moment they're found — not at wrapup, not at session end.

Qualifies: - Filter or exclusion rules found through experimentation - Numerical facts not derivable from code (row counts, limits, offsets) - Environment gotchas — args, paths, config locations, build quirks - Architecture decisions with non-obvious WHY - Any constraint requiring active investigation not visible in the code

What triggers it: Automatic. To force-capture something:

Agent	Trigger
Claude Code	`/capture <text>`
Codex / Pi	"Capture this" / "Record this in discoveries"

Layer 3 — Coding standards (always-on)

The problem: Every new session, the agent may drift from project conventions — import style, naming, git commit format — without a reminder.

What it does: The `efficiency` standard is injected into every project that has any canon skill registered. Coding principles, git conventions, token-efficiency rules. Never shown in `skills list`, needs no invocation — it just runs.

Layer 4 — Code quality

Three focused passes, each with a clear job:

`code-simplifier` — clarity and redundancy pass. Reduces nesting, eliminates dead code, improves names. Never changes behavior.

`code-reviewer` — seven-dimension review: correctness, maintainability, readability, efficiency, security, edge cases, test coverage. Reports as Critical / Improvements / Nitpicks / Recommendations.

`security-review` — high-confidence vulnerability scan. Traces data flow end-to-end before flagging anything. Only reports confirmed findings — no noisy pattern-match output.

Each step has skip logic — states why in one line when it doesn't apply:

Step	Skipped when
code-simplifier	Single-line change, or docs/config only
code-reviewer	Single-line fix with no design implications
security-review	No auth, DB, user input, API, crypto, or file I/O changed

Layer 5 — Wrapup

The problem: The three quality steps need to run in a specific order with skip logic evaluated at each step. Remembering to do this manually is friction.

What it does: `wrapup` runs all three in order, evaluates skip logic automatically, and reports a single structured summary. Inside `sprint complete`, it runs on all files modified since sprint start.

Outside a sprint: `/wrapup` directly on any code written in the session.

Layer 6 — Sprint (the full lifecycle)

What it does: Two commands encapsulate everything above:

Command	What happens
`sprint start`	Creates ticket → blueprint → acceptance criteria → reads DECISIONS.md + HANDOFF.md → produces sprint brief → **waits for your approval**
`sprint complete`	

Command	What happens
	Runs wrapup → validates every acceptance criterion → appends to DECISIONS.md → updates HANDOFF.md → closes ticket

Trigger phrases: - sprint start: any request to add, fix, update, debug, implement, or build — explicit phrases like *"sprint start"* or *"let's work on X"* also work. Skipped only for questions, explanations, or trivially mechanical one-liners. - sprint complete: *"sprint complete"*, *"approve"*, *"ship it"*

Planning files:

```
.tickets/<id>/
  ticket.md        ← tkt-managed
  blueprint.md     ← files to touch, step-by-step build plan
  acceptance.md    ← binary definition of done
```

DECISIONS.md (repo root) — durable log of non-obvious architectural choices. Sprint start reads it; sprint complete writes to it.

```
# Decisions

| Date | Decision | Reason |
|---|---|---|
| 2026-05-17 | Amounts stored as integer cents | Avoid float precision bugs |
```

Reference

Skills commands

```
$SKILLS/skills.sh list                    # show available skills
$SKILLS/skills.sh add sprint              # register a skill in current project
$SKILLS/skills.sh addall                  # register all skills (idempotent)
$SKILLS/skills.sh status                  # check registration + hook health
$SKILLS/skills.sh refresh                 # re-register + heal stale paths + prune covered deps
```

Ticket commands

Sprint manages the full lifecycle automatically. Use `tkt` directly for queries:

```
tkt ls                        # list all tickets
tkt ls --status=in_progress   # filter by status
```

```
tkt show <id>                    # full ticket detail
tkt reopen <id>                  # reopen a closed ticket
```

> Need dependency tracking, tags, or assignees? Install ticket (`brew install ticket`) — same `.tickets/` format, fully compatible.

Skill verification

Skill	How to verify	Expected response
`sprint`	`"Start a sprint for X"`	Sprint brief produced, awaits approval before any code
`pdf`	`"Extract text from [file].pdf"`	Extracted content, or a clear error
`ticket`	`tkt ls`	Empty list or existing tickets — no error

> Everything else is automatic. `efficiency` is always on. `capture` fires mid-session. `wrapup` runs inside `sprint complete`. `handoff` and `ticket` are deps of sprint — loaded silently.

Staying updated

```
cd $SKILLS && git pull
```

Hook scripts update immediately — called by path, so the new version runs on the next session.

Skill content in `CLAUDE.md` is live `@`-import references — Claude picks up changes automatically on the next session.

Inline standards in `AGENTS.md` (Codex, Pi) are a static copy — refresh explicitly:

```
$SKILLS/skills.sh refresh /path/to/your-project
```

`refresh` re-registers every skill, replaces outdated standard blocks, heals stale `@`-import paths, and prunes covered deps — all in one command.

For newly added skills:

```
$SKILLS/skills.sh addall /path/to/your-project
```

Check for issues first:

```
$SKILLS/skills.sh status /path/to/your-project
```